Lab - FIDO Webauthn

João Paulo Barraca André Zúquete

2026-02-10

1 Introduction

The goal of these exercises is to explore the functionalities of FIDO through a software token and a small web application that uses WebAuthn.

2 Using a software token

Download the Python code from the course Web page. Install the Python modules (or the correspondig Linux packages starting with 'python3-'). Run the following command:

```
$ sudo ./virtual_fido.py
```

This program emulates a FIDO2 token (authenticator), and currently implements two kinds of resident credentials: ES256 (based on the P-256 Elliptic Curve) and RS256 (based on RSA 2048).

In its default configuration (see `self.info` variable in the file `ctaphid_handler.py`), the token allows Resident Keys (RK), requires a client PIN (`1234` by default) for User Verification (UV), and allows checking the User Presence (UP). Execute the following command and check all these details (except the PIN value) in the `algorithms` and `options` attributes:

```
$ fido2-token -I /dev/hidraw1
```

You can use this token to deal with real websites, but it is not advised. Use instead the site https://webauthn.io to register credentials and authenticate with them.

Use different usernames to register different credentials. Try to use ES256 and RS256 credentials. See what happens when you allow only Ed25529.

Modify the code to disable the PIN. See what happens when you register and use credentials with different levels of user verification (discouraged, preferred, required).

3 Low-level access to WebAuthn JSON messages

The exercise will follow the example application available at https://webauthn.me/

Access the webpage, select the debug interface.

3.1 Registration

Then adjust the parameters to make a registration request with your software token (you can also use a hardware token if you have one).

You you can configure the cryptographic methods to be used. Several other parameters are required, as the need for the user to be present (User Presence) or verified (User Verification).

Hit register and create the passkey. If you have an extension such as a password manager, it may offer to store the credential. If nothing else is present, the devices available in the operating system will be used.

When the credentials are created, you can analyse what is sent to the server. In my case the response includes the following:


```
clientDataJSON: {
  "type": "webauthn.create",
  "challenge": "H_qGZ4o9bilt73C5KSVz76cV8rKTHS9q2jqTe7TxUPQ",
  "origin": "https://www.webauthn.me",
  "crossOrigin": false
}
```


```
authenticatorAttachment: cross-platform,
transports: [
  "usb"
]
```


```
attestationObject: {
  "fmt": "none",
  "attStmt": {},
  "authData": {
    "rpIdHash": "e9a88899f0e3ab9b98f693e0738a37725a3ef8130f3696b24ddddb145024880d",
    "flags": {
      "userPresent": true,
      "reserved1": false,
      "userVerified": true,
      "backupEligibility": false,
      "backupState": false,
      "reserved2": false,
      "attestedCredentialData": true,
      "extensionDataIncluded": false
    },
    "signCount": 0,
    "attestedCredentialData": {
      "aaguid": "00000000-0000-0000-0000-000000000000",
      "credentialId": "a3387e30c3cb246bc6f073ea1f7d4fb912dea9bfc80e67c7e58402207879a2db",
      "credentialPublicKey":  {
        "kty": "RSA",
        "alg": "RSASSA-PKCS1-v1_5_w_SHA256",
        "n": "qwa3wZ1ERtT+dU4MQOoqZlr8PJFgcUnxAxc5RcGmqPAPg4EsETKz1rWE0h1luaxGwVnihZkR9kvuOwFRTYAOd9BnPH\
              tnlM8l/qQmkVv3scVRg2w3Jnv/msGoCWL1yjC2jILl9/PrN6y9EGoCP+jCtXVjyjO1fu9I8tWouN92rFEPez7WnsrR\
              XlftMoWiXAJKNIk44101faeC0iHSGf9nOTJb1C0zuhBADg6M5hCR8dVkHI2vyBLg3Tc/7x0ztqXYjfhMSYWtWTIuBb\
              rsNIHUOTIP+CWRj6jfzgcHeILwhXOvqI9hYWATPxTPDdnusuKQrHva6chTNVOo2RkFJ1ehGw==",
        "e": "AAEAAQ=="
      }
    }
  }
}
```


3.2 Authentication

After authentication, the response is:


```
signature:
    9d676bc6af5f8daeb92af07242478d9f29ede7d17246372a8a960904addfab2c0cb131abcf4b241bd61b8bb4c621838abd0e\

    f0cbe398fc4056889f5b9f31daf3f8283c4e57566b7db27600b41bb2c0f7d9f24c080b641468a080eaeebfedb9ef90b84287\

    10d277a95868d6d32b09c505d8b0192331df53b62bf82a772751495d2f653a28d4b57a832e029831ae8cd5fc95519bf13d06\

    a1182f5f681c2672c845531aff83990d09190733cf0c173a5bd8e8027af3458d7bb4b55cc323b8e1a4b744827c145ec05622\
```

```
    843f03eec6c838f685a86817893c55ba718bc1979f4a3cf73022dfd3ba461371920e4a8ff05300e2db0f5afe4a50b74ef155\
          c67e40841d5a
userHandle: a9f7bd2bb12beb6e9d4dcad9d497b70ef10f0de806ff3812dbb69796e0e24ad9
clientDataJSON: {
  "type": "webauthn.get",
  "challenge": "LyDkk5ioh9z4xEm2m_N--ZnUue2jGPOhBWRS-UTXCRE",
  "origin": "https://www.webauthn.me",
  "crossOrigin": false
},
authenticatorAttachment: cross-platform,
authenticatorData: {
  "rpIdHash": "e9a88899f0e3ab9b98f693e0738a37725a3ef8130f3696b24ddddb145024880d",
  "flags": {
    "userPresent": true,
    "reserved1": false,
    "userVerified": true,
    "backupEligibility": false,
    "backupState": false,
    "reserved2": false,
    "attestedCredentialData": false,
    "extensionDataIncluded": false
  },
  "signCount": 1
},
extensions: {}
```

One point to be considered is where are the cryptographic keys actually stored. The answer will depend on the device used for authentication. Here we used a software device with volatile credentials (they are lost when the program terminates).

On Windows, you can search for `passkeys` and check the credentials already stored. The actual storage for Windows 11 is the TPM. To check the details, open a console and type: `certutil -csp NGC -key -v`